Given a Trie that stores lowercase English words, write a function:

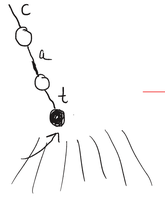
int countWordsWithPrefix(TrieNode* root, string prefix);

This function should return the number of words in the trie that start with the given prefix.
You must use a recursive helper function.

Helper Function:
int countWordsRec(TrieNode* node);

int countWordsWithPrefix(TrieNode* root, string prefix)

1) Pointer to root
2) Loop to get to a point in the trie where the prefix is
3) When you find the location of the prefix, call the recursive function to count
4) return value from helper



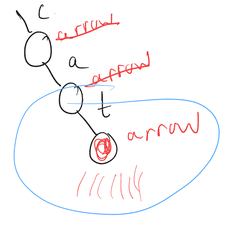
int countWordsRec(TrieNode* node)

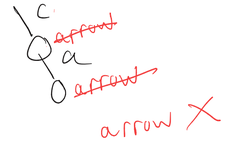
make a counter variable →

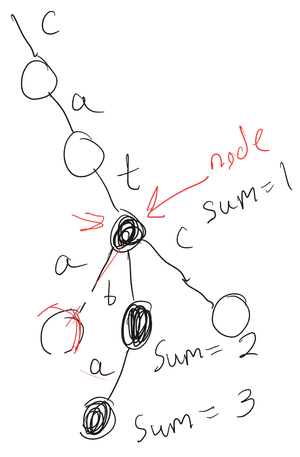
1) Base case: If node is nullptr
2) Base case: If a node is marked up our count
3) Loop through all the children recursively
4) return the count

```
int countWordsWithPrefix(TrieNode* root, string prefix){
    TrieNode* arrow = root;
    for(int i=0; i<prefix.length(); ++i){
        if(!arrow->children[prefix[i]])
            return 0;
        arrow = arrow->children[prefix[i]];
    }
    return countWordsRec(arrow);
}
```

prefix cat








```
int countWordsRec(TrieNode* node)
{
    if(!node)            // if(node==nullptr)
        return 0;
    int sum = 0;
    if(node->marked)
        sum++;
    for(int i=0; i<256; ++i){
        sum+= countWordsRec(node->children[i]);
    }
    return sum;
}
```

sum+= 0 // answer to the recursive call